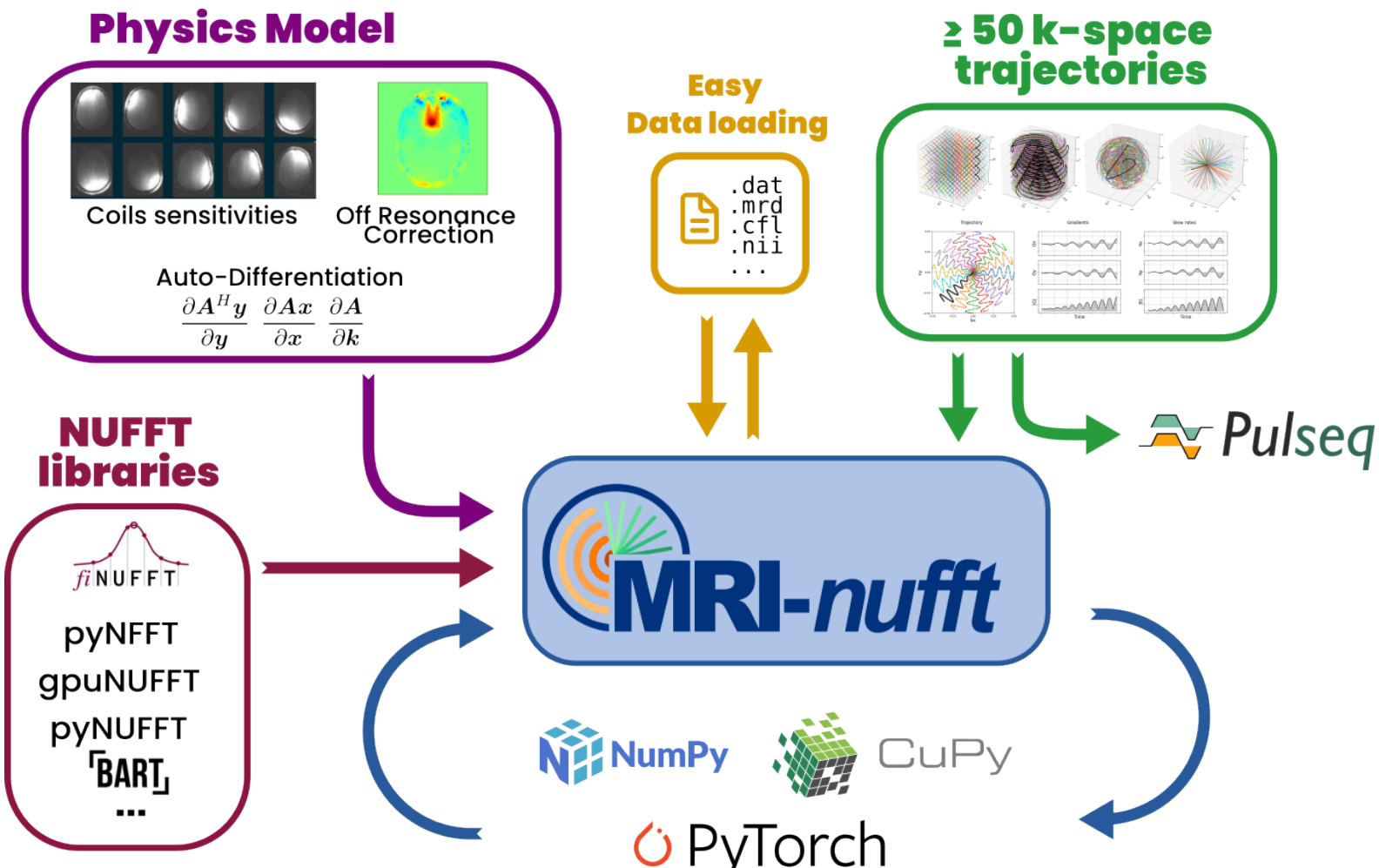
MRI-NUFFT: Symbiosis with Pulseq

2026-02-24 Pulseq Workshop

Pierre-Antoine Comby

Neurospin, CEA, Paris-Saclay University

What is MRI-NUFFT



NUFFT for MRI Reconstruction

- Inverse problem formulation for MRI

$$\hat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}\in\mathbb{C}^N} \sum_{\ell=1}^{L} \|\mathcal{F}_\Omega \mathcal{S}_\ell \boldsymbol{x} - \boldsymbol{y}_\ell\|_2^2 + \lambda g(\boldsymbol{x})$$

"Priors and Networks are temporary, …

… Forward model is for ever"

MRI-NUFFT provides *efficient* implementation for the Forward model operator:

- based on the NUFFT backend of your choice.
- Handles CPU/Device transfers and array conversion (`torch/numpy/cupy`)
- `autograd` support $\frac{\partial \boldsymbol{A}\boldsymbol{x}}{\partial \boldsymbol{x}}, \frac{\partial \boldsymbol{A}^{\boldsymbol{H}}\boldsymbol{y}}{\partial \boldsymbol{y}}, \frac{\partial \boldsymbol{A}_\theta \boldsymbol{x}}{\partial \theta}, \ldots$

```
from mrinufft import get_operator
nufft = get_operator("cufinufft", samples, shape, smaps=smaps, density="pipe")
ksp = nufft.op(image)          # y = Ax
adjoint = nufft.adj_op(ksp)    # x = A^H y
pinv = nufft.pinv_solver(ksp)  # x = A^† y, using CG
```

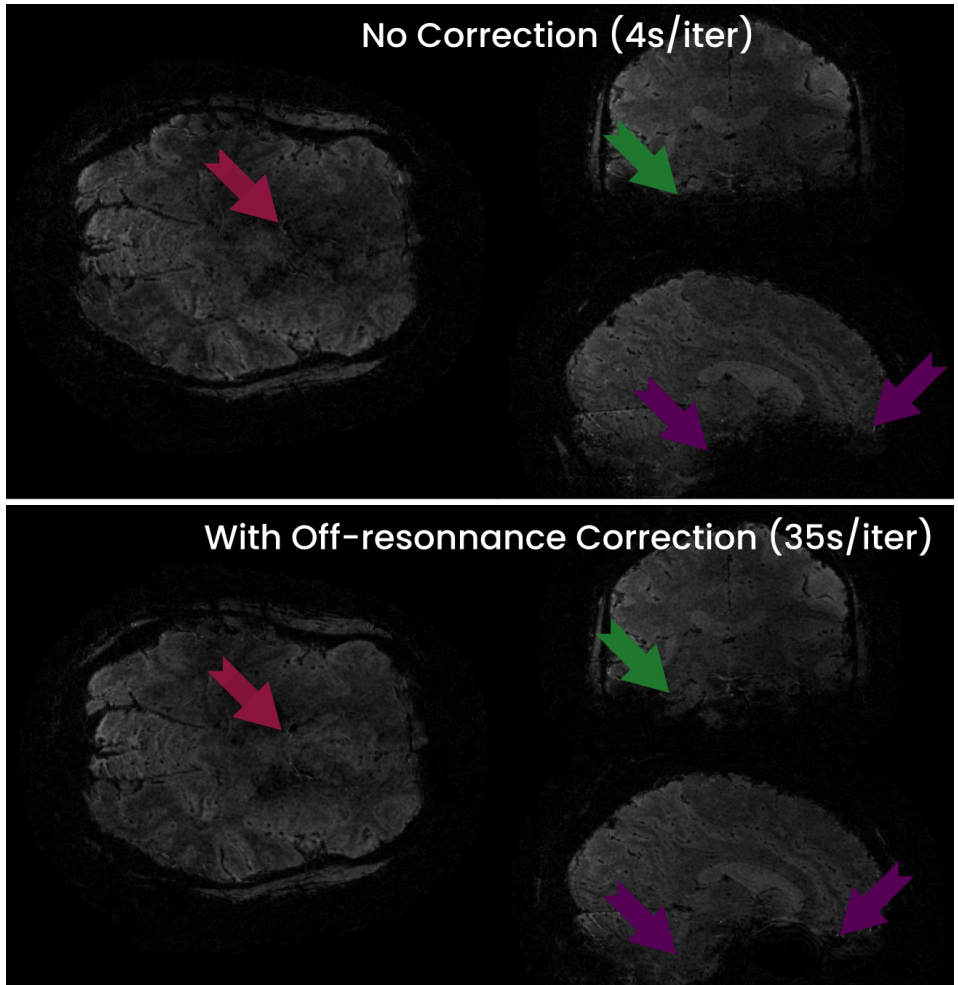
Extended Fourier Model

- Multi-coil, subspace representations, …
- Density Compensation (`voronoi`, `pipe`, …)
- Off Resonance and bilinearization (MTI, MFI, **SVD**)

$$\boldsymbol{y}_\ell(t) = \int_{FOV} \boldsymbol{x}(\boldsymbol{r})\mathcal{S}_\ell(\boldsymbol{r})e^{-\Delta\omega(\boldsymbol{r})t}e^{-2i\pi\boldsymbol{k}(t)\cdot\boldsymbol{r}}\mathrm{d}\boldsymbol{r}$$

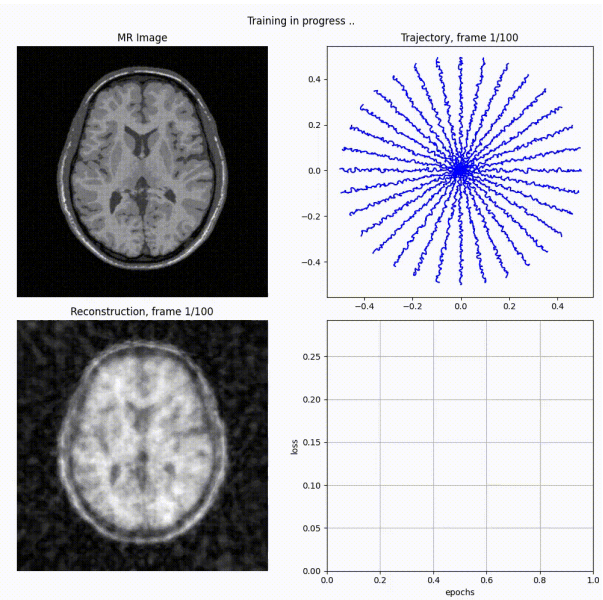
$$\min_{\substack{\boldsymbol{B}\in\mathbb{C}^{M\times P}\\ \boldsymbol{C}\in\mathbb{C}^{P\times N}}} \|e^{-\Delta\omega(\boldsymbol{r})t} - \boldsymbol{B}\boldsymbol{C}\|_2^2 \Longrightarrow \boldsymbol{y}_\ell \approx \sum_{p=1}^{P}\boldsymbol{b}_p\sum_{\ell=1}^{L}\mathcal{F}_\Omega(\boldsymbol{c}_p\mathcal{S}_\ell\boldsymbol{x})$$

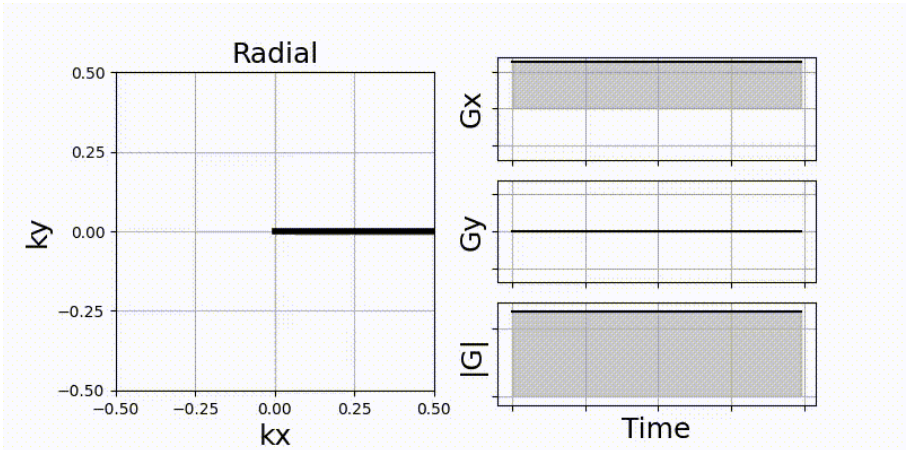
```
nufft_orc = nufft.with_off_resonance_correction(
    field_map=field_map,
    readout_time=readout_time,
    interpolators=20)
adj_orc = nufft_orc.adj_op(ksp) # x = A^H_Δω y
pinv_orc = nufft_orc.pinv_solver(ksp) # x = A^†_Δω y
```

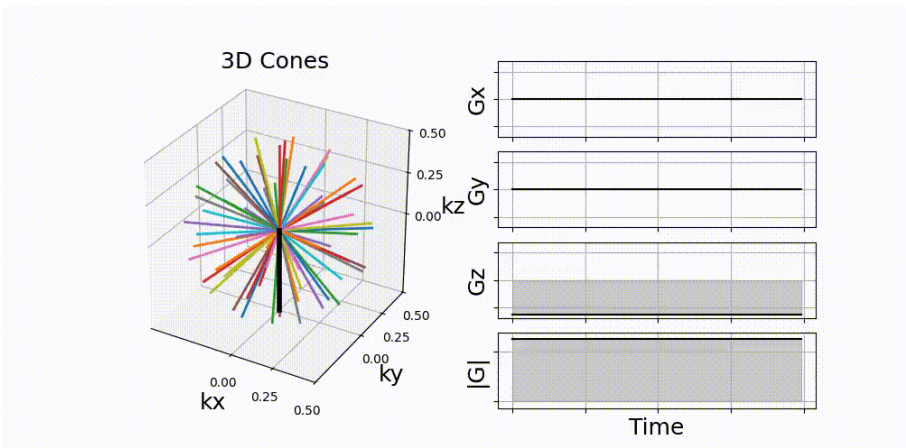


Trajectories & Gradients

- A large collection of non-Cartesian trajectories
  - 2D and 3D, fully parameterized
  - You can even learn trajectories in a deep Learning setting with autodifferentiation !







Harware compliant trajectories: Projection

Trajectories Constraints

Set of admissible trajectories

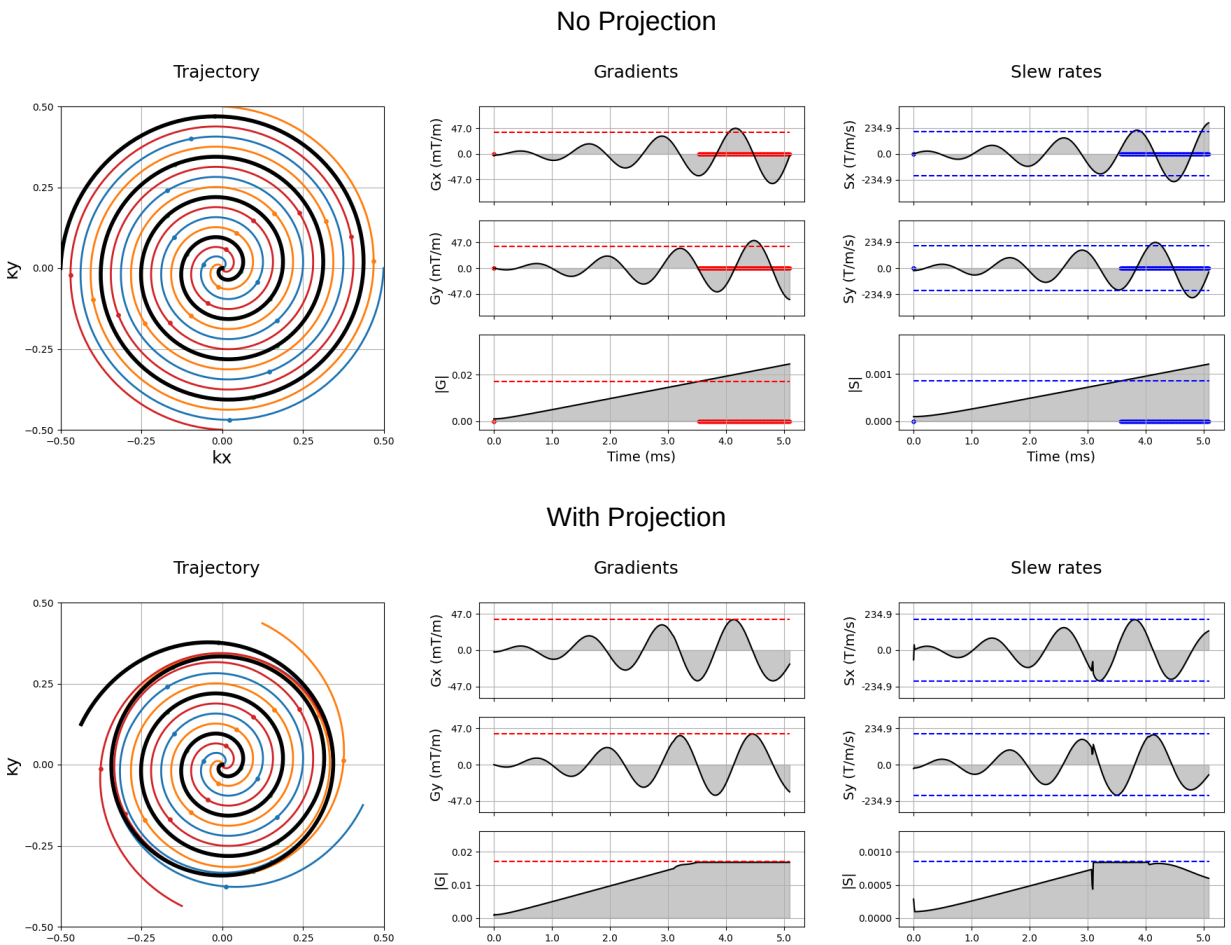
$$\mathcal{Q} = \left\{ \begin{array}{c} \forall i = \{1, ..., N_c\}, \boldsymbol{k}_i \in \Omega^{N_s} \\ \|\boldsymbol{k}\|_\infty \leq \frac{1}{2}, \|\dot{\boldsymbol{k}}\|_{2,\infty} \leq g_{\max}, \|\ddot{\boldsymbol{k}}\|_{2,\infty} \leq s_{\max} \\ \boldsymbol{A}_i \boldsymbol{k}_i = \boldsymbol{b}_i \end{array} \right\}$$

Given an existing trajectory $\boldsymbol{k} \in \Omega^{N_c \times N_s}$:

$$\boldsymbol{k}' = \arg\min_{\boldsymbol{k}' \in \mathcal{Q}} \| \boldsymbol{k} - \boldsymbol{k}' \|_2$$

Solved with a Primal Dual Algorithm (Chauffert et al., 2016; Giliyar Radhakrishna et al., 2023)

```
from mrinufft.trajectories import Acquisition,
SIEMENS
acq = Acquisition(
    hardware = SIEMENS.TERRAX,
    TE=0.050, TR=0.100, FA=12,
    fov=(.192,.192,.129), img_size=(64,64,43))
new_traj = project_trajectory(traj, acq=acq)
```



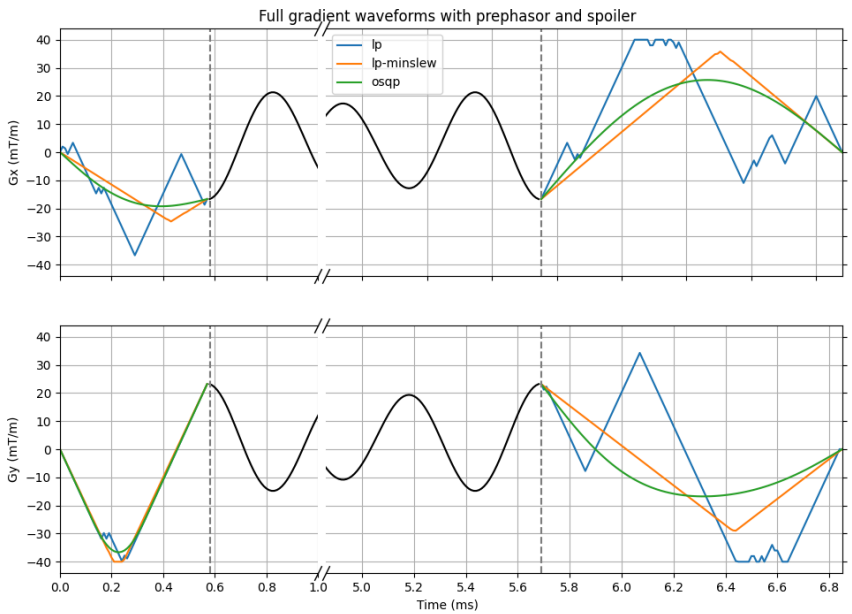
Optimal Prephasors and Rewinders

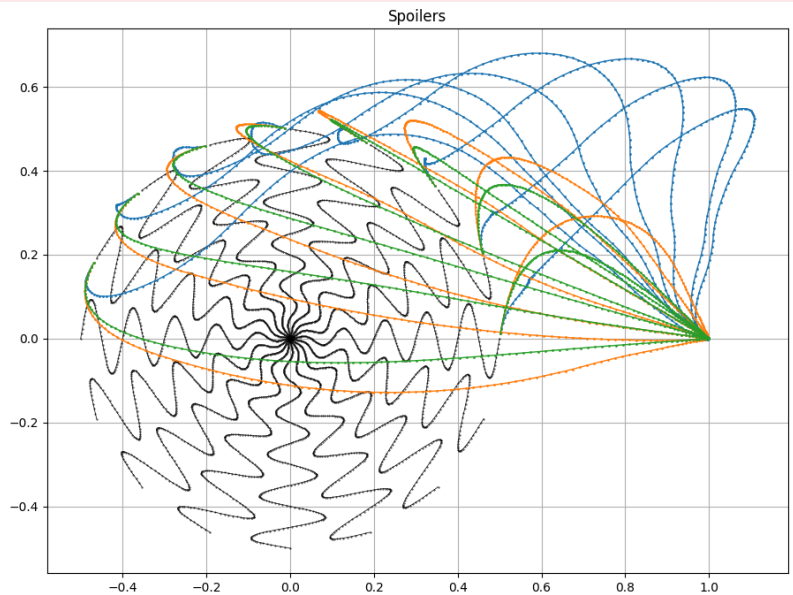
- Non-Cartesian trajectories need to start at an arbitrary point in k-space, but usually we start at the center
  - Increase the non-Cartesian gradients with ADC-off segments

Linear and Quadratic Programing

Given $\|\dot{k}\|_\infty \leq g_{\max}$, $\|\ddot{k}\|_\infty \leq s_{\max}$, Δk, g_{start}, g_{end}

1. What is the minimum of raster time points required?
2. Given this number of points, what is the path to take?
3. Make the trajectory as smooth as possible: minimize curvature of gradients $\|\dddot{k}\|$





Integration with Pulseq

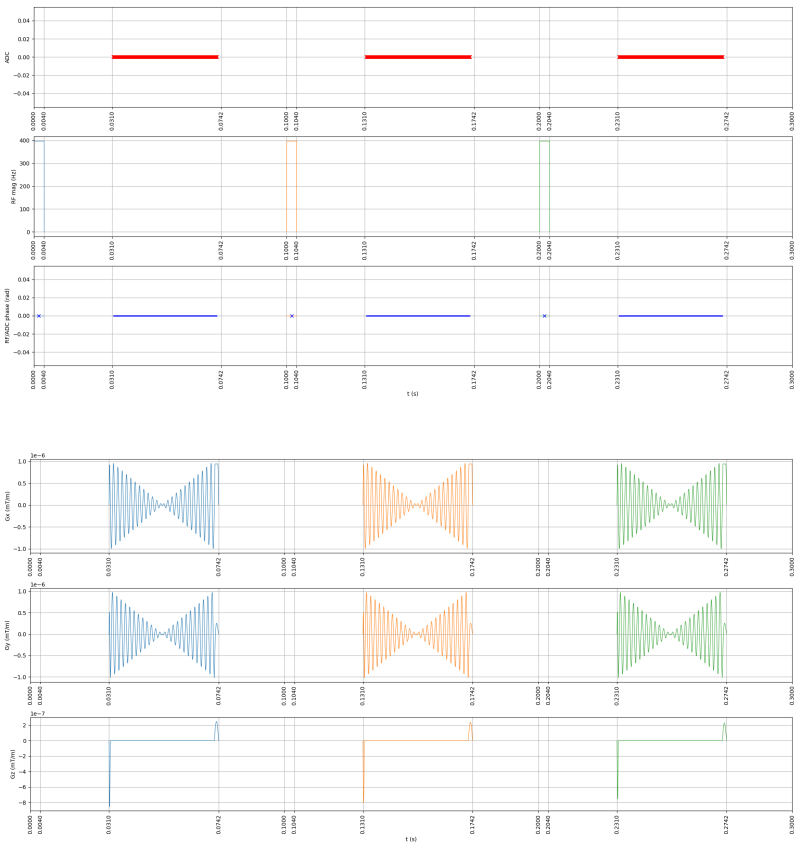
Current State

- Hardware Compliant trajectories via projection
- Optimal prephasors/spoilers

- Neurospin runs on Siemens hardware
- We have a custom “Arbitrary gradient” sequence, works well but not portable to other systems. → **Pulseq!**

```
from mrinufft.trajectories import pulseq_gre, read_pulseq_traj

traj = ...
seq = pulseq_gre(traj, acq=acq)
seq.write("spiral_gre.seq")
traj_read = read_pulseq_traj(seq)
```



Chronogram of the Pulseq Sequence

Conclusion

You can use MRI-NUFFT for:

- **Fast and efficient NUFFT** operators for non-Cartesian MRI reconstruction
  - To be used inside your reconstruction libraries
- Testing a **large collection** of ready-to-use **non-Cartesian trajectories**
- Augmenting, and adapt/constraint these trajectories.
- Design with `(Py)Pulseq`.

https://github.com/mind-inria/mri-nufft

https://mind-inria.github.io/mri-nufft

Comby et al., (2025). *MRI-NUFFT: Doing non-Cartesian MRI has never been easier*. Journal of Open Source Software, 10(108), 7743, https://doi.org/10.21105/joss.07743

Thank you
for your attention

pierre-antoine.comby@cea.fr

Neurospin, CEA, Paris-Saclay University